Transient Feature Editing with Sparse Autoencoders: What Reaching Rank 1 Does and Does Not Show, Within One Model and Across Two

Adithyan R S Alona Biju Alvin Binu Darsan M
Department of Computer Science and Engineering (AI)
Mar Baselios College of Engineering and Technology (Autonomous)
Thiruvananthapuram, India

Abstract

Language models often rank a correct answer below a generic competitor even though the answer is within reach. Weight edits fix this permanently, and sequential weight editing is known to degrade (Gupta et al., 2024). We ask whether the repair can instead be *transient and weight-free*: during one forward pass the residual stream of GPT-2 Small is edited through the features of a pretrained sparse autoencoder (SAE), and the edit is discarded afterwards. We document the construction of such an edit, including the ideas that failed, and evaluate it on 300 held-out CounterFact prompts. Tuning one multiplier per SAE feature by gradient descent brings the true answer to rank 1 on 271 prompts, against 161 for a hand-built sweep given the same strength range and 94 for the original sweep, with lower side effects on the prompt and 5 to 6.5 times less compute. A control then shows that rank 1 is a weak test: an additive variant forces random, unrelated words to rank 1 on 20 of 20 prompts. We calibrate our metrics against ROME's official implementation, and on ROME's paraphrase score our edit generalises less than a weight edit (67% against 98.5%, on different records), consistent with the edit being a push toward a word. Finally, we ask whether the change an edit makes in GPT-2 Small can be carried into GPT-2 Medium through a translator fitted on ordinary text, with every pass bar fixed before its run. The translator keeps the receiving model working, and an exported edit lifts the target's median rank from 142 to 14, but reaches rank 1 on only 8% of records; every map that does not look at the receiver's output stops near 20%, while per-sentence tuning against the receiver reaches 95%. What crosses is a push toward a word, not a fact. In the other direction, mixing half of Medium's translated state into Small raises Small's accuracy on held-out facts from 8.5% to 9.5%. Code, result files and logs are public.

1 Introduction

Language models often finish a factual prompt with a generic word although the right answer sits a few places down the ranking. Given "The location of Massachusetts Institute of Technology is in", GPT-2 Small (Radford et al., 2019) puts " the" first and " Cambridge" eighth (Table 1). We call this *suppressed factual recall*; the name describes the output ranking only and makes no claim about what the model "knows".

The usual remedies change the model. Fine-tuning and closed-form weight edits such as ROME (Meng et al., 2022a) and MEMIT (Meng et al., 2023) are permanent, and when they are applied one after another they first wear away earlier edits and then break the model (Gupta et al., 2024). Retrieval and in-context editing (Lewis et al., 2020; Zheng et al., 2023) leave the weights alone but hand the fact to the model as extra text. Mechanistic interpretability suggests a third option. A sparse autoencoder (SAE) (Bricken et al., 2023; Cunningham et al., 2024) rewrites a layer's activations in a much wider basis in which few directions are active at a time, and these *features* often line up with single concepts. A feature is a direction in the residual stream, so it can be turned down (*muted*) or up (*boosted*) during one forward pass and the change thrown away afterwards. If that works, an error can be repaired *transiently*, with no weight touched, and the repair can be read as a short list of named features.

This paper reports what happened when we tried it, in the order it happened, with the failures kept. Our contributions are:

1. A transient edit of causally ranked SAE features with a per-feature safety filter, whose design choices are each backed by a measurement

Source	Prompt (true answer / counterfactual target)
CounterFact	"The mother tongue of Danielle Darrieux is" (*French* / *English*)
known_1000	"The Eiffel Tower is located in" (*Paris* / *Rome*)
Probe prompt	"The location of Massachusetts Institute of Technology is in" (*Cambridge*, rank 8 behind " the" and six more words)

Table 1: Example prompts. The target is a single token; "rank" is its place in the next-token ordering of the unedited model.

(Sections 3 and 4).

2. An equal-budget comparison on 300 held-out CounterFact prompts: per-feature gradient descent brings the true answer to rank 1 on 271, against 161 for a sweep with the same strength range and 94 for the original settings, with 5 to 6.5 times less compute (Section 5).
3. A control showing that rank 1 is a weak test: an additive edit pushes random words to rank 1 on 20 of 20 prompts, so success at rank 1 cannot be read as recovered knowledge (Section 6).
4. A calibration of our metrics against ROME's official code on GPT-2 Small (Section 7).
5. A cross-model study with every pass bar written before the run: a linear translator between GPT-2 Small and Medium, Export and Import tests with controls, a documented search that locates the limit of Export at the compatibility of the edit rather than at the channel, a significance test showing that what crosses is a push, and a first net-positive Import result (Section 8).

2 Setting, Models and Metrics

Models. "Small" is GPT-2 Small (12 layers, residual width 768) with the pretrained SAE release `gpt2-small-res-jb` (24,576 features per layer; Bloom, 2024), loaded through TransformerLens and SAELens (Bloom et al., 2024; Nanda and Bloom, 2022). All edits act at the hook `blocks.L.hook_resid_pre` with $L = 8$ unless stated. "Medium" is GPT-2 Medium (24 layers, width 1024) with no SAE; it shares Small's tokenizer.

ROME	Question	This paper
ES	New answer beats the true one on the edit prompt	Target first over the whole vocabulary
PS	The same on reworded prompts	"Rewordings", same definition
NS	True answer still wins on neighbour prompts	"Neighbours", same definition

Table 2: ROME's scores and their counterparts here.

Data. CounterFact (Meng et al., 2022a) pairs each prompt with the true answer and a counterfactual one and adds reworded and neighbouring prompts. Of its 21,919 records, 16,360 have a single-token true answer that Small ranks between 2 and 1000. We split them so that a subject appears in one set only and five relations are held out, giving 1,000 training, 150 validation and 300 test prompts, and cache layer-8 states, candidate features and safety verdicts for 1,450 of them. All choices (layer, dose, epoch) are made on development data; test prompts are only read.

Metrics. For a prompt with true answer t, the *rank* of t is its place in the next-token ordering over the whole vocabulary, and the *blocker* b is the token ranked first. An edit *succeeds* when t is first; this is stricter than ROME's efficacy score, which compares only the new and the old answer. We also report the *same-prompt KL* (Eq. 8), the *edit size* (norm of the change over the residual norm at the last position), and the effect on unrelated prompts, on reworded prompts and on generated text. ROME's *paraphrase score* (PS: share of reworded prompts where the new answer beats the true one) and *neighbourhood score* (NS: share of other-subject prompts where the true answer still beats the new one) are used with their original definitions (Table 2). Methods are compared on identical prompts with the exact McNemar test (McNemar, 1947) on reaching rank 1 and the Wilcoxon signed-rank test (Wilcoxon, 1945) on KL; bootstrap intervals (Efron, 1979) are used for differences of shares. *Controls* accompany every claim: a random word of matched starting rank as target, another record's edit, a random vector, and, where stated, a plain push along the target's output direction.

3 The Edit

SAE features and delta patching. Let $x_p \in \mathbb{R}^{768}$ be the residual stream at position p. The SAE gives sparse feature activations and a reconstruction,

$$f_p = \mathrm{ReLU}\big(W_{\mathrm{enc}}(x_p - b_{\mathrm{dec}}) + b_{\mathrm{enc}}\big), \qquad \hat{x}_p = W_{\mathrm{dec}}^{\top} f_p + b_{\mathrm{dec}}. \tag{1}$$

Replacing x_p by $\hat{x}_p$ would inject the reconstruction error (12 to 26% of the activation norm) and disturb the model even with no edit. We therefore add only the change in the decoded output. If feature k is scaled by s_k ($s_k = 1$ leaves it alone),

$$\Delta_p = \sum_k (s_k - 1)\, f_{p,k}\, W_{\mathrm{dec}}[k], \qquad x'_p = x_p + \Delta_p \tag{2}$$

at every prompt position. Muting with strength $\mu \in [0, 1]$ is $s_k = 1 - \mu$; boosting with $\beta \geq 0$ is $s_k = 1 + \beta$. With every $s_k = 1$ the edit is exactly zero.

Causal ranking. Ranking all 24,576 features by removal would cost as many forward passes. Stage 1 keeps the top N features by activation, at the last token or over all prompt positions except the start token. Stage 2 removes each candidate alone and records

$$\Delta p_k(v) = P_{\mathrm{ablate}(k)}(v) - P_{\mathrm{clean}}(v). \tag{3}$$

The most negative values for the blocker ($v = b$) are its strongest drivers (the *mute list*); for the target ($v = t$), its strongest helpers (the *boost list*). The N removals are stacked into one batched call, so ranking costs two calls instead of $2N+1$.

Safety filter and sweep. A candidate is kept only if it does not hurt the target when applied alone:

$$r_k(t) \leq r_0(t) \;\wedge\; p_k(t) - p_0(t) \geq -10^{-6}, \tag{4}$$

with r, p the target's rank and probability with feature k applied alone and r_0, p_0 the baseline. The *hybrid sweep* applies the first k mutes and k boosts together, for $k = 1, 2, \ldots$, and reports the best step rather than the last, because softmax coupling means more editing is not always better. When the pools run dry, *pool refill* keeps the features applied so far, takes the steered model's new blocker and re-ranks the remaining features against it (Figure 16, Appendix C).

Gradient-descent editor. The sweep chooses which features and how many, with one strength per side. The gradient-descent editor instead gives each of $K = 200$ candidate features its own multiplier, $m_k = 1 + a_k$ with free numbers z_k initialised so that nothing changes:

$$a_k = -1 + 3\,\sigma(z_k), \qquad m_k \in (0, 3), \tag{5}$$

$$\Delta_p = \sum_{k=1}^{K} a_k\, A_{p,k}\, W_{\mathrm{dec}}[k], \tag{6}$$

where $A_{p,k}$ is the clean activation of feature k at position p. We run blocks 8 to 11 from the edited state and read the logits ℓ at the last position. The loss is

$$\mathcal{L} = \max\big(0, \gamma - (\ell_t - \max_{v \neq t} \ell_v)\big) + \lambda_{\mathrm{KL}} \mathbf{1}[t \text{ leads}]\, \mathrm{KL} + \lambda_{\mathrm{size}} \sum_k |a_k| w_k, \tag{7}$$

with margin $\gamma = 0.3$, $\lambda_{\mathrm{KL}} = 3$, $\lambda_{\mathrm{size}} = 0.005$ and w_k the feature's share of the total maximum activation. The first term asks the target to beat every other token by γ logits; the second penalises changes to the rest of the distribution,

$$\mathrm{KL} = \sum_{v \notin \{t,b\}} q_v \log \frac{q_v}{\tilde{p}_v}, \tag{8}$$

where q and $\tilde{p}$ are the softmax of the clean and edited logits restricted to tokens other than t and b, so the intended change is not penalised; the third prefers small edits. Only the z_k are updated, by Adam (Kingma and Ba, 2015) (learning rate 0.1, 100 steps); GPT-2 and the SAE are frozen. We keep the best iterate and re-check it through the same hook as the sweep in a full forward pass, so the reported rank does not depend on the shortcut used during tuning (Figure 17, Appendix C).

The additive edit. A multiplier cannot switch on a silent feature, since $0 \times m = 0$. The additive extension adds an amount $c_k \geq 0$ of the decoder direction of selected silent features at the last position,

$$\Delta_{\mathrm{last}} \mathrel{+}= \sum_{k \in \mathcal{S}} c_k\, W_{\mathrm{dec}}[k], \tag{9}$$

where $\mathcal{S}$ holds the 200 features that are exactly silent at the last position and have the largest first-order score $W_{\mathrm{dec}}[k] \cdot g$, with $g = \partial(\ell_t - \max_{v \neq t} \ell_v)/\partial x_{\mathrm{last}}$. Each c_k starts at zero and is

capped at the loudest feature on the real prompt positions. Because the score is a gradient of the target's margin, this edit writes the *output direction of the answer*, not a representation of a fact (Section 6).

4 How the Design Was Reached

This section condenses the trial-and-error record (journal entries 1 to 30). The early experiments used one prompt or at most 131 hand-made prompts that share templates; they explain the design and are not performance estimates.

Ablation and competitors. Ablating the features that *help* the target (strength 0.3, up to five rounds) corrected 1 of 22 suppressed facts, because muting what supports the target weakens it. Muting the drivers of the *competitor* corrected 3 of 22, all from rank 2, and failed whenever the leader was a generic word such as " the" (Figure 1a). On the probe prompt the seven competitors collapsed onto two features (313 for " the", " a", " an"; 21169 for " question", " jeopardy", " doubt"), yet muting them left the target at rank 8; raising the strength on feature 313 by 73% moved the target's probability only from 1.29% to 1.39%. For each competitor, support was spread over many features (the second feature 92 to 95% as influential as the first).

Mute and boost together. Three mutes plus three boosts brought the probe prompt to rank 1; a fourth boost pushed it back to rank 2 (an isolated rerun reproduced the same 6.7357%), because that feature supports " the" more than " Cambridge" (Figure 1b). The lesson, to report the *best* step, became part of the algorithm.

Where to edit, and speed. A layer sweep on 12 eligible prompts showed a plateau at layers 6 to 9 with layer 8 highest (mean rank gain 7.33, against 7.08, 6.50 and 6.33 at layers 7, 6 and 9, and at most 4.75 elsewhere); we kept layer 8 as a default, not a proven optimum (Figure 2). A cost audit found 186 model calls per prompt and layer. Removing a baseline recomputed inside every candidate loop saved 60 calls, and stacking candidates into the batch dimension brought the number to 6 (21-fold) with identical ordered feature lists, probabilities (to 10^{-9}) and all 60 safety decisions. A GPU gave a further factor 3.8, and a full hybrid run fell from about 12 s to under 1 s (Figure 18, Appendix C).

Reliability, candidate supply and the safety rule. The sweep initially reported its last step instead of its best, chained mute and boost as two hooks (rank 66 against 59 for a combined check) and stopped early because its pool only held features that passed the filter against the *clean* model. Pool refill fixed the last problem and exposed a hard limit: for "The leading cause of death in men is" only 67 features were active at the last token, and scaling cannot activate a feature that is off (the run used all 67 and stopped at rank 51, from 152). Drawing candidates from *all* prompt positions raised the prompts at rank 1 from 4 to 10 of 13 comparable prompts (won 9, lost 0; exact sign test $p \approx 0.004$; Figure 3). We then asked whether the strict safety rule was too strict. In a 45-prompt pilot, removing the filter lost 10 of 29 successes and gained none; a 5% tolerance lost 2; graded strengths changed nothing. On 131 last-token prompts relaxed rules added at most one prompt, and only 2.5 to 2.9% of features were rejected on both sides. The supply of features, not the filter, was the limit, and the strict filter stayed.

Is it repaired by later layers, or limited by the SAE? On prompts that had failed in every mode, the pushed direction was still 0.79 to 1.29 of its injected size at the output, and re-adding whatever fraction had been lost gave the same rank in 21 of 21 layer and prompt cells: we saw no repair. A free residual edit of the same size reached rank 1 in all 21 cells against 1 for the SAE edit. Because we chose these prompts for having failed, this is a yardstick, not a success rate. A literature check the same week found mixed evidence on SAE steering (Section 9) and led us to compare every later result with controls.

Headroom and learned strengths. A headroom measurement on the 1,450 cached prompts asked how much room any choice of multipliers has. Free per-feature multipliers could bring the target to rank 1 on 80% of prompts if side effects are ignored, and on 41% with a side-effect penalty, against 6 to 15% for crude fixed rules (Figure 4). The sweep uses one mute strength (0.6) and one boost strength (0.5). We trained networks to choose them: *PromptNet* reads 12 summary numbers about the prompt and

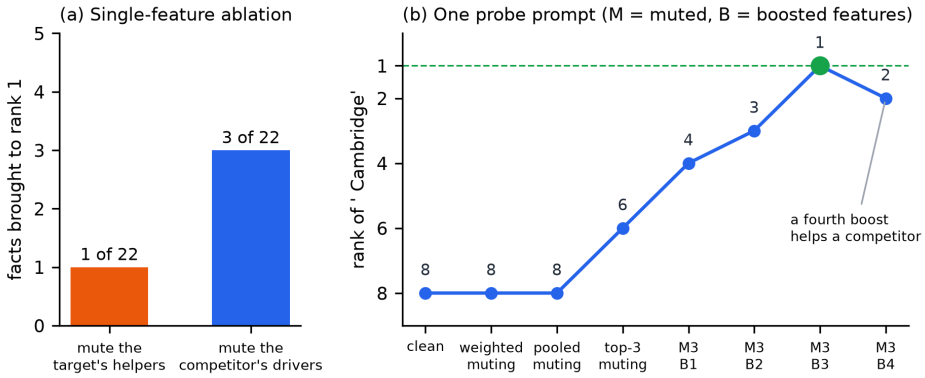


Figure 1: Early experiments. (a) 22 suppressed facts: muting the target's own helpers corrected 1, muting the competitor's drivers corrected 3. (b) Probe prompt: muting alone stayed at rank 8, mute and boost reached rank 1, a fourth boost undid it.

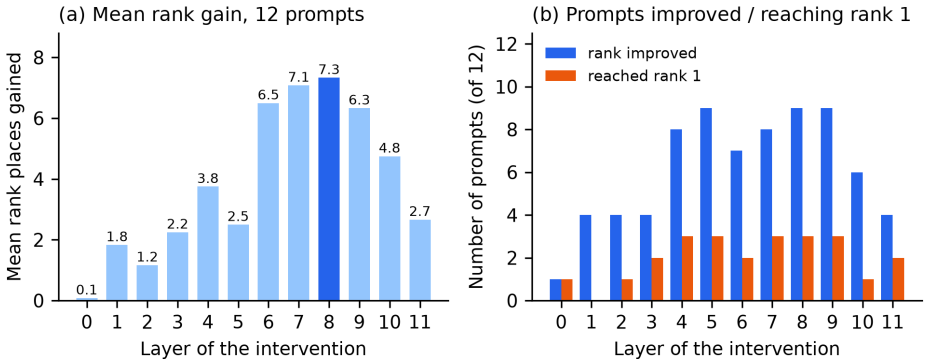


Figure 2: Effect of the intervention layer on the 12 eligible prompts. (a) Mean rank places gained by the target. (b) Prompts whose rank improved (blue) and reached rank 1 (orange). Layers 6 to 9 form a plateau; layer 8 (dark bar) has the highest mean gain.

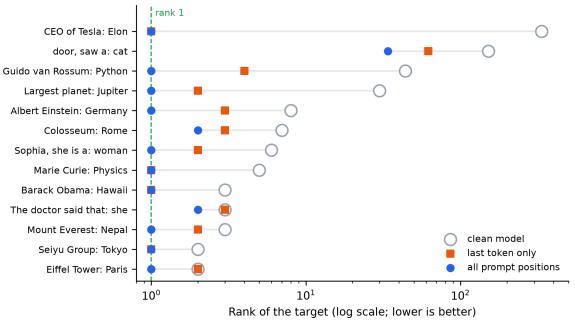


Figure 3: Final rank of the target on the 13 comparable prompts with candidates from the last token only or from all prompt positions (log scale; rank 1 is the dashed line).

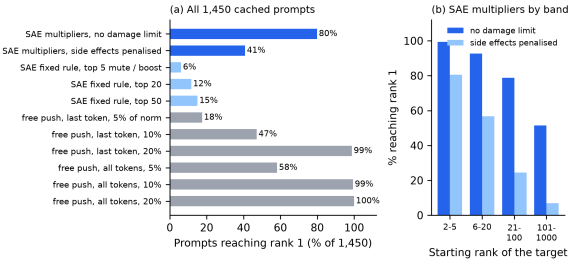


Figure 4: Headroom on the 1,450 cached prompts (one-shot edits on saved states, a stand-in for the real sweep). (a) Share reaching rank 1 by method. (b) Free multipliers by starting-rank band.

outputs two strengths; *FeatureNet* outputs one multiplier per candidate. Under a stand-in measure on the 300 test prompts, fixed 0.6/0.5 reached rank 1 on 75 (25.0%), one learned pair shared by all prompts (mute 0.98, boost 1.81) on 116 (38.7%), PromptNet on 119 and a per-prompt oracle on 123 (Figure 18, right). The gain came from a better *global* pair; choosing per prompt added 3 prompts (exact McNemar $p = 0.25$), roughly doubled side effects, and FeatureNet (31 to 32%) stayed below the pair. The network was not the useful part; its *teacher* was: free multipliers tuned on each prompt reached 73% in the same stand-in, nearly twice the network. This suggested using the optimiser itself as the editor.

Kept	Dropped, and why
Delta patching (no reconstruction error)	Muting the target's helpers: weakens the target
Causal ranking by ablation, batched	Weighted, summed or top-K muting alone: support is spread
Mute and boost, strict per-feature filter	Relaxed or graded filters: ≤ 1 prompt gained
Best-step reporting, pool refill, all-position candidates	Last-token candidates: features run out
Gradient-descent multipliers	Learned strength network: no gain per prompt
Controls, held-out benchmark, fixed pass bars	Additive edit as a fix; "knowledge absent" label

Table 3: What the trial-and-error record kept and what it dropped.

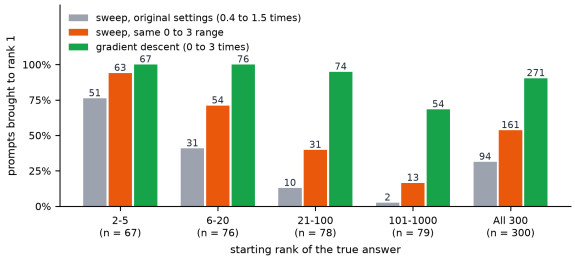


Figure 5: Prompts brought to rank 1 by the starting rank of the true answer, 300 held-out prompts: original sweep, sweep with the same 0 to 3 range, and gradient descent.

5 Gradient Descent against the Sweep

Our first hand trials favoured gradient descent but were unfair, because it could scale features from 0 to 3 times while the sweep ranged only from 0.4 to 1.8. The controlled comparison therefore gives the sweep the same range (mute 1.0, boost 2.0) and measures all arms with one yardstick through the real hook.

On the 300 held-out prompts, gradient descent brought the true answer to rank 1 on **271** (90%), the wide sweep on 161 and the original sweep on 94 (Figure 5). It solved 110 prompts the wide sweep missed and lost none. Its same-prompt KL was lower on 160 of the 161 prompts that both solved, and it needed about 7.7 s per prompt against 38.5 s and 49.7 s, i.e. 5 to 6.5 times less compute (Figure 6). Two cautions apply. Part of the KL advantage is built in, since its loss contains a KL term and the sweep has none; and the median target probability at rank 1 is only 7.7%, so success means "first", not "confident".

6 Rank 1 Is a Weak Test

The 29 prompts gradient descent failed on are mostly deep targets whose helpful features may be silent. The additive edit (Eq. 9) can switch such features on, and its first hand trials looked excellent: all six hard prompts reached rank 1, including absurd targets such as a random name after "Hello, my name is". We then paired 20 validation prompts with random, unrelated words. Multipliers reached rank 1 on 1 of 20; the additive edit reached it on **20 of 20**, at an edit 126 to 192% the size of the residual stream itself (Figure 7). This is the analogue of fitting random labels (Zhang et al., 2017) or of an adversarial perturbation (Goodfellow et al., 2015; Szegedy et al., 2014): with enough freedom, an optimiser reaches any output requested. The edit also carried over to reworded prompts about as well for a random word as for the true answer (median rank 4 against 4.5; rank 1 on 25% against 16%) and changed the top word on 82% of unrelated prompts. It steers toward a word and does not repair a fact.

Three consequences followed. Rank 1 is a weak test, so every later result carries controls. Being able to steer a target to the top says nothing about knowledge in either direction, so we withdrew an early "knowledge absent" label. And the additive edit is off by default, with a warning: it answers "can we force GPT-2 to say what we want?" with "almost anything", which is exactly why it is not a fix.

Beyond the next word. We tested the multiplier edit on 40 test prompts (10 per starting-rank band). Tuned on one prompt and applied unchanged to its *rewordings*, it lowered the true answer's median rank from 94 to 36 and reached rank 1 on 9%, while a random word went from 17,504 to 11,832 and reached rank 1 on none: a modest fact-related part, but mostly local (Figure 8). On 20 *unrelated* prompts the multiplier edit changed the top word on 6%, the additive edit on 17 to 24%. For *generated text* (20 greedy tokens, edit kept on) fluency was unchanged (perplexity 4.4 to 4.8 against 4.7), variety dropped slightly (distinct 2-grams 0.85 to 0.89 against 0.93; Li et al., 2016), loops appeared in 2 to 12% of continuations against none (Holtz-

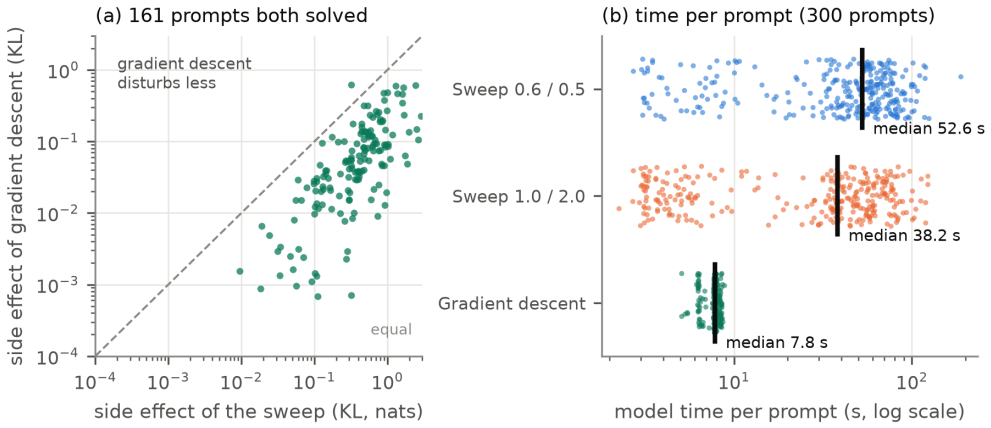


Figure 6: Gradient descent against the sweep with the same range on the 300 held-out prompts. (a) Same-prompt KL for the 161 prompts both solved; gradient descent is lower on 160 (points below the dashed line). (b) Model time per prompt, log scale; medians 52.6 s (original sweep), 38.2 s (wide sweep), 7.8 s (gradient descent).

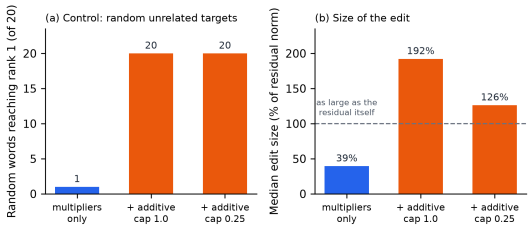


Figure 7: Control with random unrelated targets: the additive edit reaches rank 1 on 20 of 20 prompts at a size above the residual stream's; multipliers on 1 of 20.

man et al., 2020), and the target was repeated 0.6 to 0.7 extra times. For *instructions*, with the prefix "Do not say the word X", the unedited model already put X first on 45% of prompts (a named word primes copying); the multiplier edit raised this to 68% and the additive edit to 88 to 90%. The edit ignores the instruction.

7 ROME as an External Reference

To check that our numbers are not private to our code, we reproduced the official ROME implementation (Meng et al., 2022a,b) on GPT-2 Small, traced where its facts are decided, and scored our edit with its metrics. ROME treats an MLP layer as a key-value memory (Geva et al., 2021): with key k_* at the last subject token and a value v_* found by gradient descent that makes the model say the new answer, it rewrites the second MLP matrix W in closed form,

$$\hat{W} = W + \Lambda\,(C^{-1}k_*)^\top, \quad \Lambda = \frac{v_* - Wk_*}{(C^{-1}k_*)^\top k_*}, \tag{10}$$

with $C = \mathbb{E}[kk^\top]$ the second moment of keys over ordinary text. The change is permanent; ours acts on the residual stream, lasts one pass, and is chosen by a sweep or by gradient descent.

Causal tracing. In causal tracing (Meng et al., 2022a; Pearl, 2001; Vig et al., 2020) the subject's tokens are corrupted and the clean state is restored one token and layer at a time. We traced the 97 facts (of 342 that GPT-2 Small completes correctly among 1,209 in ROME's known-facts file) with clean answer probability at least 0.2 (mean 0.358, falling to 0.034 under corruption). Recovery is almost entirely at the last prompt token in the upper layers (residual stream 0.32 at layer 11, attention up to 0.10 at layer 10); at the last subject token, where ROME writes, the MLP effect is small (+0.01 at layers 0 and 1, about -0.02 at layers 2 and 3) (Figure 9). For GPT-2 Small there is thus no sharp mid-layer MLP site of the kind reported for GPT-2 XL, and tracing does not fix the edit layer. The late last-token effect lies in the same range as our layer plateau (6 to 9); the two come from different interventions, so this supports the range but does not prove the choice.

Edit layer. ROME was run at layers 1, 3, 5, 7 and 9 on 100 CounterFact records (seed 0). Efficacy is 100% at every layer, so layers differ only in damage to neighbours (Figure 10). Layer 3 has the best overall score ($S = 0.851$) and was chosen, though layers 1 to 5 are within noise at this sample size; even there the neighbour score falls from 0.716 to 0.663. An edit takes about 4 s on a CPU.

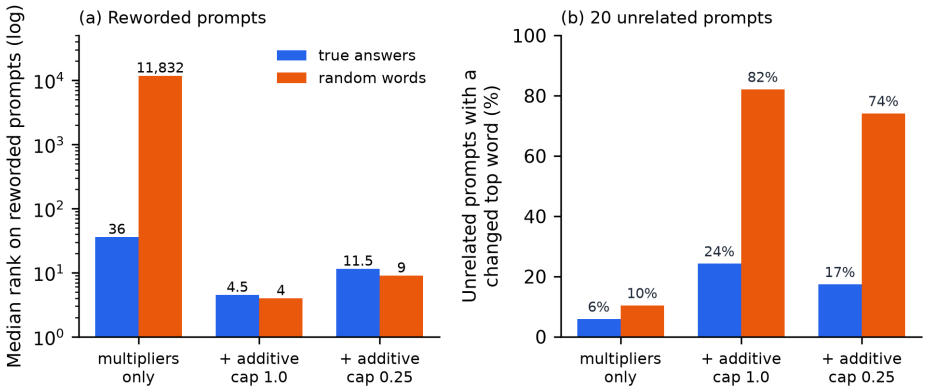


Figure 8: Effect on other prompts with the true answer (blue) or a random word (orange) as target, for multipliers and two additive edits (cap 1.0 and 0.25). (a) Median target rank on reworded prompts (log scale): 36 for the true answer and 11,832 for a random word under multipliers; 4 to 12 for both under additive edits. (b) Share of 20 unrelated prompts whose top word changed.

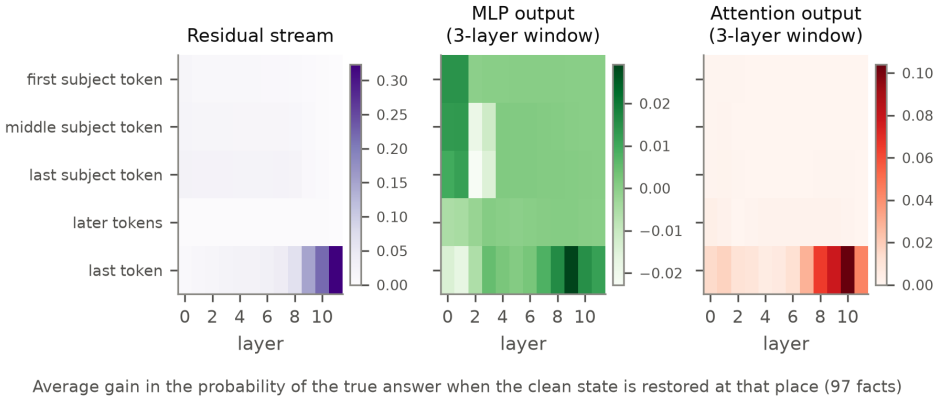


Figure 9: Average causal trace of GPT-2 Small over 97 known facts: gain in the probability of the true answer when the clean state is restored at one token role (rows) and layer (columns). The effect is concentrated at the last token in layers 8 to 11.

Cross-check. Our gradient-descent edit can be scored with PS and NS because the controlled Export study (Section 8) tuned it in GPT-2 Small toward each record's counterfactual answer and re-applied the same multipliers to reworded and neighbour prompts. ROME's layer-3 edit lifts PS from 33% to 98.5% and loses 5 points of NS; ours lifts PS from 40% to 67% and loses 9.5 points (88 test records on which the edit reached rank 1; Figure 11). Unedited levels are in the same range (PS 33% and 40%, NS 72% and 63%), so our measurement of the same quantity is not out of line with the official code. The weaker carry-over to rewordings agrees with the finding that our edit is mostly a push. The record sets differ and the two edits are re-applied differently (ROME changes the weights once; our multipliers are recomputed per text), so the comparison is indicative only. A head-to-head comparison with ROME, MEMIT or in-context editing on one frozen record list has *not* been run (Section 11).

8 Cross-Model Transfer of an Edit

Once the edit, the controls and the benchmark existed, we applied them to a new question: *can the change an edit makes inside GPT-2 Small be translated and added to GPT-2 Medium at inference time with no weight change, and can Medium's state improve Small?* We call the first direction **Export** and the second **Import** (Figure 12). The models do not share coordinates (width 768 against 1024), so a translator is needed. This is a side study: the SAE is used only where the edit is made, and Medium has none.

8.1 Design

Small's layer 8 is paired with Medium's layer 16. Of the 200 benchmark records used here (50 devel-

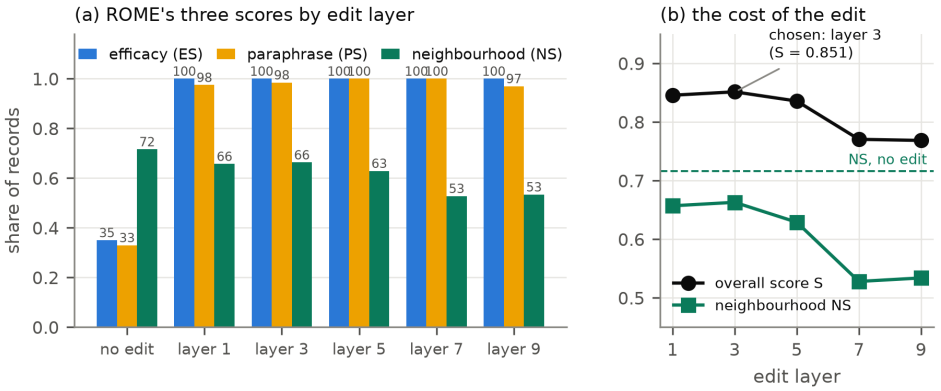


Figure 10: ROME edit-layer sweep on GPT-2 Small, 100 development records. (a) ES, PS and NS for the unedited model and each edit layer. (b) Overall score S and neighbourhood score by layer; the dashed line is the unedited NS.

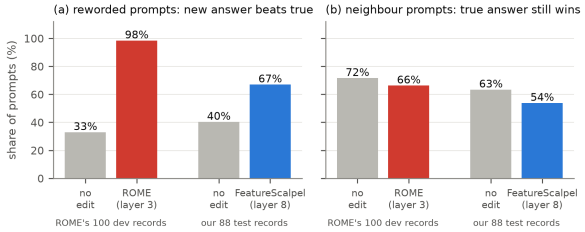


Figure 11: Paraphrase (PS) and neighbourhood (NS) scores before and after an edit: ROME at layer 3 (100 development records) and our gradient-descent edit at layer 8 (88 test records). Indicative only.

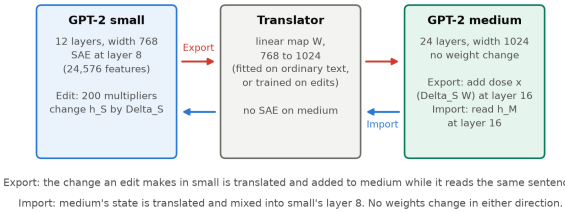


Figure 12: The two directions studied. A translator maps one model's residual stream into the other's coordinates; no weights change.

opment, 150 test), the edit tuned in Small reaches rank 1 on 116; the **88 primary test records** are the test records where it does, since only there is there an edit to carry. Medium's median target rank on them is 142 before anything is done.

Translators. Only the *change* an edit makes is translated, so a bias cancels and the translated change is $\Delta_S W$ (zero at the first position). We tried four ways to get W. (1) A *ridge map* (Hoerl and Kennard, 1970) fitted on paired states of the two models over Wikitext-2 (Merity et al., 2017) (500,126 fit tokens), $W = \arg\min \|(H_S - \mu_S)W - (H_M - \mu_M)\|^2 + \lambda\|W\|^2$. (2) A *neural map*: the ridge map plus a one-hidden-layer correction. (3) A *linear map trained on edits*, started from the ridge map and trained so that Medium's answers move as Small's do, with target distribution $q = \mathrm{softmax}(\ell_M + \ell_S^{\mathrm{edit}} - \ell_S)$ and loss $\mathrm{KL}\big(q \,\|\, p_M(h_M + \alpha\Delta_S W)\big)$ over 289 successful training edits with all dev and test target words held out. (4) *No fixed map*: the 200 multipliers are retuned for each sentence against *Medium's* output, a ceiling and not a transfer, since it looks at Medium every time.

Export, Import and controls. Export adds the translated change while Medium reads a sentence, $h'_M = h_M + \alpha\Delta_S W$ at layer 16 at every position but the first. The **dose** α is chosen on development records; from step D10 on we choose it by the best mean log-rank gain, because dev top-1 on 28 records proved fragile (Section 8.4). Import mixes Small's state with Medium's translated state, $h'_S = (1-\gamma)h_S + \gamma T(h_M)$, with $\gamma = 1$ full replacement. The receiver is never shown the answer. Export controls are a random vector of equal size, another record's edit, an edit toward a random word of matched starting rank, a *word push* along the target's output direction (which uses the answer, so it is a comparison and not a method) and the fact stated in Medium's prompt; Import controls are a state from a wrong sentence, the average state and a random map.

Pass bars. Every pass bar ("gate") was written into the project plan and committed before the run that judged it, and none was moved. A step designed after seeing a result is labelled post-hoc and cannot change a verdict. The bars and verdicts are in Appendix B.

Arm	Top-1	Rank	PS	NS
Unchanged	0%	142	36%	67%
Translated edit	**8%**	**14**	**52%**	**63%**
Random vector	0%	157	38%	66%
Other record's edit	0%	128	36%	65%
Word push*	83%	1	82%	23%
Fact in prompt	–	–	97%	44%

Table 4: Export of the edit through the ridge map (88 primary test records; dose chosen on dev). Rank is the target's median rank; PS and NS are the rewordings and neighbours scores. *Uses the answer.

8.2 The Channel Works

The ridge map fits well enough to keep the receiver working: per-dimension R^2 is 0.58 from Small layer 8 to Medium layer 16 and 0.66 from Medium 16 to Small 8, and stitching one model's first layers to the other's last layers through the map recovers 91% (Export) and 98% (Import) of the receiver's own quality. It copies *which word changed* well (cosine 0.63 to 0.76) but the summary at the last position only weakly (0.24 to 0.31), so Gate 1 passes for Import and fails for Export. As a positive control we swap one country for another in a template sentence, translate the difference between the source model's states, add it to the receiver, and count a hit when the receiver's top answer becomes the swapped capital: **55%** (Export) and **33%** (Import) against at most 0.4% for a random vector, a random map and another pair's difference (Gate 2 passes). This controls the channel; it is not an edit and not a fact.

8.3 Export: Real but Weak

For each of the 88 primary test records we pass the multiplier edit tuned in Small through the ridge map and add it to Medium at layer 16 (Table 4). The effect is real, graded and specific: the target's median rank falls from 142 to **14** and it becomes the top answer on **8%** of records; reworded prompts favour the new answer on 52% against 36% unchanged; neighbours lose 4 points; the random vector and another record's edit do nothing. A word push makes the target first on 83% of records but only 23% of neighbours keep the true answer, and stating the fact in Medium's prompt costs 23 points of neighbours. The bar (top-1 $\geq$ 15% and 10 points above the strongest control) is not met (Gate 3 fails).

8.4 Searching for a Better Translator

Seven changes were tried, each with its rule written first (Figure 13). Injecting at Medium layers 4, 8, 12, 16 and 20 gave 0, 5, 5, 8 and 1%, and splitting the dose over several layers did no better (best layer set 7% against 8% for the best layer; Gate 4 fails), so the layer is not the limit. A neural map fits ordinary text much better (R^2 up by 0.09 to 0.11) but exports no better (6 to 8%): a better fit to text does not help. A **map trained on edits** roughly doubled the result to 18% top-1 (median rank 14 down to 7, paired $p = 5 \times 10^{-9}$) and stayed specific to the record, but fell short of the bar (Gate 6). More training edits (13, 13, 20% at 25, 50, 100% of the data), longer training (no reproducible change; the apparent drop to 2% was the dose chosen on 28 dev records, since at dose 2.0 both settings gave 20%) and a direct rank-1 hinge term in the loss (16 to 21% for every weight; Gate 7 fails) left it at about 20%.

Why it plateaus. With the rank-1 term the map learned its 289 training targets (95% top-1 on them) but not unseen words (16%): it memorises. Gate 9 then tested whether data variety was the limit: 1,221 additional edits toward 759 random target words stopped the memorisation (training–test gap 12 to 3 points) but unseen records got *worse*, 20% to 14%. The loss asks Medium to *copy Small's shift* and not to reach rank 1 (the KL to Small's shifted distribution was equal, 0.082 against 0.085), so a better copy does not help.

Per-sentence tuning is a ceiling. Re-tuning the multipliers against Medium's output through the translator reaches the top answer on 95% of the 88 records (84% of all 150), with the same neighbour cost as the source edit inside Small (8 points against 9). The channel can therefore carry a rank-1 edit, and what fails is the compatibility of an edit tuned for Small with Medium. The ceiling's solution points only partly in the direction of the original edit (cosine 0.51 against 0.02 by chance), so a map that merely transforms the given edit cannot supply what is missing.

8.5 What Is Carried: a Fact or a Push?

The gain on reworded prompts must be measured against each arm's *own* unchanged level, since a

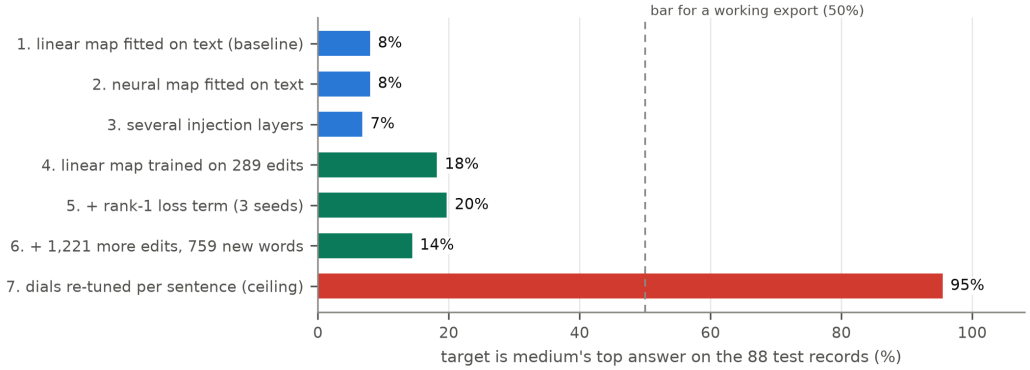


Figure 13: Top-1 of every export variant on the same 88 test records. Blue: map fitted on text. Green: map trained on edits. Red: per-sentence tuning against Medium's output (a ceiling, not a transfer).

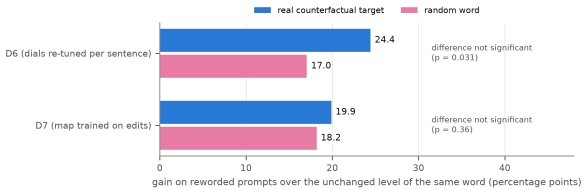


Figure 14: Gain on reworded prompts over each word's own unchanged level, for the real counterfactual target (blue) and a random word (pink), 88 test records. Per-sentence tuning (D6): $+7.4$ points ($p = 0.031$); map trained on edits (D7): $+1.7$ ($p = 0.36$). Neither passes the $p < 0.01$ bar.

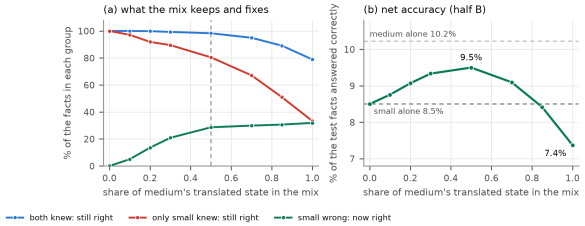


Figure 15: Import by mixing Small's own state with Medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy, best at a 50/50 mix and below Small alone under full replacement.

random word starts lower (25% against 36%). Measured this way, per-sentence multipliers gain 24.4 points for the real target and 17.0 for a random word (difference $+7.4$, 95% bootstrap interval 0.0 to $+15.3$, $p = 0.031$), and the map trained on edits gains 19.9 and 18.2 ($p = 0.36$) (Figure 14). Neither difference passes the $p < 0.01$ line fixed beforehand. An earlier statement that the real target's lift was "four times" the random word's compared the random word with the *real* target's baseline and is withdrawn. What carries to rewordings is a push toward the targeted word, not something specific to the fact. In a further control we repeated the per-sentence tuning with 200 random directions in place of the 200 SAE directions (same firing pattern): the SAE directions reached 95% and the random ones 81%. The SAE makes the tuning more effective but is not what makes steering possible.

8.6 Import: Medium's State into Small

We use the 21,789 single-token CounterFact facts left after removing development and pilot records. Medium alone gets 9.9% right and Small 8.3%; 780 facts are right in Medium and wrong in Small. **Full replacement** of Small's layer-8 state by Medium's translated state rescues 262 of the 780 (33.6%; controls at most 0.8%), so real, sentence-specific information crosses. But it breaks 22% of the facts both models knew and 70% of those only Small knew, and Small's accuracy falls from 8.30% to 7.17% (Gate 8 fails).

A **softer mix**, with its rule written before it was built, chose $\gamma \in \{0.1, \ldots, 1\}$ on one half of the facts and reports the other half (10,895 facts). Mixing in half of Medium's state ($\gamma = 0.5$) raises Small's accuracy from 8.5% to **9.5%** (Medium alone: 10.2%): 162 facts gained, 53 lost, $p = 2.5 \times 10^{-14}$, 98% of the facts both models knew kept, 28.6% of Small's misses rescued (Figure 15). Gating by Medium's confidence added nothing. The rule (gain ≥ 0.5 points, 90% of known facts kept, $p < 0.01$) is met, but the gain is one point, it needs Medium to run on the same sentence, and controls for the mix were not rerun.

9 Related Work

Knowledge editing. ROME (Meng et al., 2022a) locates a fact in middle MLP layers and rewrites one matrix in closed form; MEMIT (Meng et al., 2023) extends this to thousands of edits. Both are

permanent. Gupta et al. (2024) apply up to 2,000 sequential edits to GPT-2 XL and GPT-J and find two phases of forgetting: ROME's efficacy stays near 100% until a per-sample inflection point, after which previously edited facts and downstream task performance collapse together with a spike in the distance between the edited layer's weights and their originals; MEMIT delays this point. Hase et al. (2023) find that where a fact is stored does not necessarily tell where it is best edited. Retrieval (Lewis et al., 2020) and in-context editing (Zheng et al., 2023) change the input rather than the model's computation, and toolkits such as EasyEdit (Wang et al., 2024) standardise the three ROME metrics we reuse. Our transient edit is motivated by this scaling problem: nothing persists after the forward pass, so edits cannot accumulate.

Locating facts. Feed-forward layers act as key-value memories (Geva et al., 2021); causal mediation analysis (Pearl, 2001; Vig et al., 2020) underlies causal tracing (Meng et al., 2022a); activation patching is surveyed by Heimersheim and Nanda (2024) and gradient shortcuts such as AtP* by Kramár et al. (2024). The residual stream is the basic object of this view (Elhage et al., 2021).

Steering and SAEs. Activation Addition (Turner et al., 2023), contrastive activation addition (Rimsky et al., 2024) and representation engineering (Zou et al., 2023) add directions to the residual stream. Superposition (Elhage et al., 2022) explains why neurons rarely match concepts; SAEs (Bricken et al., 2023; Cunningham et al., 2024; Templeton et al., 2024) learn interpretable features, with open releases for GPT-2 Small (Bloom, 2024) and Gemma (Lieberum et al., 2024). Several studies edit SAE features: Chalnev et al. (2024) aim steering vectors at features, Zhao et al. (2025) edit a few features to control memory versus context, Farrell et al. (2024) unlearn via ablation, and Marks et al. (2025) use features as circuit nodes. Arad et al. (2025) show that selecting features by output effect improves steering two to three times, the idea behind our causal ranking. Evidence is mixed: AxBench finds simple baselines can beat SAEs at steering (Wu et al., 2025), a DeepMind update reports negative downstream results (Smith et al., 2025), SAEs struggle against a neuron baseline on GPT-2 Small factual knowledge (Chaudhary and Geiger, 2024), and behaviour suppressed by clamping SAE features can return through the part of the activation the SAE does not explain (Cui et al., 2026). None tests exactly our setting; together they are why we use controls.

Gradient-based optimisation of edits. ROME itself finds its value vector by gradient descent with a term that keeps the subject's meaning (Meng et al., 2022a); our loss has the same two ingredients but tunes SAE multipliers and changes no weight. Soft prompts and prefix tuning (Lester et al., 2021; Li and Liang, 2021) and optimised steering vectors (Dunefsky and Cohan, 2025; Subramani et al., 2022) show that a frozen model can be steered through a small optimised input; adversarial examples (Goodfellow et al., 2015; Szegedy et al., 2014) and random-label fitting (Zhang et al., 2017) are the right comparison for our additive edit. As far as a limited search could tell, none tunes one multiplier per *SAE feature* for a single prompt with a side-effect term and compares it with a hand-built sweep on held-out prompts.

Stitching and transfer. Model stitching joins lower layers of one network to the upper layers of another through a small learned layer (Bansal et al., 2021; Lenc and Vedaldi, 2015); CKA (Kornblith et al., 2019) compares activations without stitching. Chen et al. (2025) show that an affine map between residual streams of two language models, including GPT-2 Small and Medium, moves features across at low cost: they carry SAEs, probes and steering vectors, but not fact edits. Our translator is a ridge regression of the same family and we do not claim it as new; what is new here is testing whether an *edit's effect* crosses, with controls and fixed pass bars.

10 Discussion

What the main line shows. The sweep was held back by the supply of features, not by its selection rule: candidates from all prompt positions raised the prompts at rank 1 from 4 to 10 of 13, and choosing the right features and strengths mattered more than stronger interventions. Tuning one multiplier per feature then did far better (271 of 300), and the oracle in the learned-strength study was the hint that the optimiser, not a network that imitates it, should be the editor. At the same time rank 1 turned

out to be a weak test: the additive edit forces random words to the top, the edit carries little beyond the prompt it was tuned on, and it mostly ignores an instruction not to say the word. Our edit is a push toward a word, and the ROME cross-check (PS 67% against 98.5%) is consistent with that.

What the side study shows. A linear channel between GPT-2 Small and Medium exists and keeps the receiver working. The change an edit makes in Small moves Medium's answers in a specific, graded way without damaging neighbours, but every translator that does not look at Medium's output stops near 20% top-1, far below the 50% bar, while looking at Medium for every sentence reaches 95%. Together with the controls this places the limit on the *edit's compatibility* and not on the channel, the layer, the translator's capacity or the amount of data. Mixing in half of Medium's state is the one positive result, and it is small. We expect, but have not tested, that the mismatch is largely a property of the edit being tuned on Small's own output.

Habits that paid off. Controls first, fair comparisons (the same strength range for the sweep), pass bars fixed before each run, and corrections made in the open: three withdrawn claims ("knowledge absent", "mainly the subject's representation", and the "four times" rewordings contrast) are marked where they happened. Choosing a hyper-parameter by top-1 on a very small dev set was fragile, so the dose rule changed after step D9b to mean log-rank gain.

11 Conclusion

We built a transient, weight-free edit of GPT-2 Small through the features of a pretrained SAE and showed that tuning one multiplier per feature by gradient descent brings the true answer to rank 1 on 271 of 300 held-out CounterFact prompts, far more often and more cheaply than a hand-built sweep. Controls then showed that rank 1 does not mean recovered knowledge: the same machinery pushes random words to the top, the edit is a push toward a word, and an exported edit carries a push and not a fact. A linear translator carries the *direction* of an edit into GPT-2 Medium, but no translator that does not look at Medium's output reaches the pass bars (about 20% top-1 against 50%); per-sentence tuning against Medium reaches 95%, so the limit is the edit's compatibility with the receiver. In the other direction, a half-weight mix of Medium's translated state raises Small's accuracy by one point. The results support the claim that a transient SAE-feature edit can raise a suppressed answer to the top. They do not support the claims that such edits recover knowledge, that they are specific to a fact, or that they beat weight editing, in-context editing or prompting.

Limitations

We use one model family, one older SAE release and layer 8 in all later studies. The early studies use small templated prompt sets (13 to 131 prompts), treat prompts as independent and do not correct for multiple comparisons. Success means rank 1 on the next token (median target probability at rank 1 is 7.7%); side effects were measured with one seed, without confidence intervals, and with true targets only in the generation and instruction tests. The headroom and learned strengths are stand-ins measured on cached states, and we never ran the real sweep with learned strengths. The transfer study has 88 primary test records (one record is about one point) and a 28-record dev set, one model pair, one dataset, one SAE and one fixed split for Import; seeds at 100% training data share their data; the hold-out rule left only 289 successful training edits; the neighbour reference of Gates 5 and 6 was fixed after seeing step D6, and several later steps (D9b, the dose rule) are labelled post-hoc. The random-word rewordings and neighbour rows printed for steps D8 to D13 use the real target's unchanged level as baseline and are uncorrected (no verdict depends on them). The ROME cross-check uses different record sets for the two edits. **We have not compared the approach with ROME, MEMIT, in-context editing, prompting or difference-of-means steering**, so nothing here says how it ranks against them; ROME-edited GPT-2 Medium as a second donor was not run. Import was tested on single-token facts only, without controls for the mix, rewordings, other relations or partial positions. The additive edit, generated text, other model pairs and larger models were not tested in the transfer study. Our related-work descriptions of some sources (notably Chen et al., 2025) rest on their abstracts and should be checked against the full papers before any novelty claim.

Future work. (i) A specificity benchmark that reports, for every method on the same prompts, rank 1, rewordings, unrelated prompts, generated text and instruction-following, with random-word controls. (ii) A head-to-head with ROME, MEMIT, in-context editing and difference-of-means steering, timed on one machine. (iii) Rerunning the learned strengths in the real sweep, controls for the Import mix, and other model pairs and larger models with pretrained SAEs on both sides (Lieberum et al., 2024).

Code and data. Code (`src/`, `tools/`, `tools/transfer/`), numbered research-journal entries, result files and raw logs are in the repository `https://github.com/adi5022/Mechanistic-Intervention-of-Large-Language-Models-via-Sparse-AutoEncoders` (branch `cross-model-transfer`); Appendix D lists the commands. Trained maps and per-record caches (about 1 GB) are not in git.

References

D. Arad, A. Mueller, and Y. Belinkov. 2025. SAEs are good for steering – if you select the right features. In *Proc. EMNLP*. arXiv:2505.20063.

Y. Bansal, P. Nakkiran, and B. Barak. 2021. Revisiting model stitching to compare neural representations. In *NeurIPS*, vol. 34.

J. Bloom. 2024. Open source sparse autoencoders for all residual stream layers of GPT2 small. *AI Alignment Forum*.

J. Bloom, C. Tigges, A. Duong, and D. Chanin. 2024. SAELens. GitHub repository.

T. Bricken, A. Templeton, J. Batson, et al. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*.

S. Chalnev, M. Siu, and A. Conmy. 2024. Improving steering vectors by targeting sparse autoencoder features. arXiv:2411.02193.

M. Chaudhary and A. Geiger. 2024. Evaluating open-source sparse autoencoders on disentangling factual knowledge in GPT-2 small. arXiv:2409.04478.

A. Chen, J. Merullo, A. Stolfo, and E. Pavlick. 2025. Transferring linear features across language models with model stitching. In *NeurIPS*. arXiv:2506.06609.

M. Cui, L. Shen, and X. Yang. 2026. SAE interventions are unreliable: Post-intervention recovery of suppressed behavior. arXiv:2606.18322.

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. 2024. Sparse autoencoders find highly interpretable features in language models. In *ICLR*.

J. Dunefsky and A. Cohan. 2025. One-shot optimized steering vectors mediate safety-relevant behaviors in LLMs. In *COLM*. arXiv:2502.18862.

B. Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

N. Elhage, N. Nanda, C. Olsson, et al. 2021. A mathematical framework for transformer circuits. *Transformer Circuits Thread*.

N. Elhage, T. Hume, C. Olsson, et al. 2022. Toy models of superposition. *Transformer Circuits Thread*.

E. Farrell, Y.-T. Lau, and A. Conmy. 2024. Applying sparse autoencoders to unlearn knowledge in language models. arXiv:2410.19278.

M. Geva, R. Schuster, J. Berant, and O. Levy. 2021. Transformer feed-forward layers are key-value memories. In *Proc. EMNLP*, pages 5484–5495.

I. J. Goodfellow, J. Shlens, and C. Szegedy. 2015. Explaining and harnessing adversarial examples. In *ICLR*.

A. Gupta, A. Rao, and G. Anumanchipalli. 2024. Model editing at scale leads to gradual and catastrophic forgetting. In *Findings of ACL 2024*. arXiv:2401.07453.

P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. 2023. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. In *NeurIPS*, vol. 36.

S. Heimersheim and N. Nanda. 2024. How to use and interpret activation patching. arXiv:2404.15255.

A. E. Hoerl and R. W. Kennard. 1970. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67.

A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. 2020. The curious case of neural text degeneration. In *ICLR*.

D. P. Kingma and J. Ba. 2015. Adam: A method for stochastic optimization. In *ICLR*.

J. Kramár, T. Lieberum, R. Shah, and N. Nanda. 2024. AtP*: An efficient and scalable method for localizing LLM behaviour to components. arXiv:2403.00745.

S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. 2019. Similarity of neural network representations revisited. In *ICML*.

K. Lenc and A. Vedaldi. 2015. Understanding image representations by measuring their equivariance and equivalence. In *CVPR*.

B. Lester, R. Al-Rfou, and N. Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proc. EMNLP*.

P. Lewis, E. Perez, A. Piktus, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, vol. 33.

J. Li, M. Galley, C. Brockett, J. Gao, and B. Dolan. 2016. A diversity-promoting objective function for

neural conversation models. In *Proc. NAACL*, pages 110–119.

X. L. Li and P. Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proc. ACL*.

T. Lieberum, S. Rajamanoharan, A. Conmy, et al. 2024. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. arXiv:2408.05147.

S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller. 2025. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *ICLR*. arXiv:2403.19647.

Q. McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

K. Meng, D. Bau, A. Andonian, and Y. Belinkov. 2022. Locating and editing factual associations in GPT. In *NeurIPS*, vol. 35, pages 17359–17372.

K. Meng et al. 2022. ROME: Rank-One Model Editing, official implementation. GitHub repository, commit 0874014 (MIT licence).

K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. 2023. Mass-editing memory in a transformer. In *ICLR*.

S. Merity, C. Xiong, J. Bradbury, and R. Socher. 2017. Pointer sentinel mixture models. In *ICLR*.

N. Nanda and J. Bloom. 2022. TransformerLens. GitHub repository.

J. Pearl. 2001. Direct and indirect effects. In *Proc. UAI*, pages 411–420.

A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Technical Report*.

N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner. 2024. Steering Llama 2 via contrastive activation addition. In *Proc. ACL*, pages 15504–15522.

L. Smith, S. Rajamanoharan, A. Conmy, et al. 2025. Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update #2). *LessWrong*.

N. Subramani, N. Suresh, and M. E. Peters. 2022. Extracting latent steering vectors from pretrained language models. In *Findings of ACL 2022*, pages 566–581.

C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. 2014. Intriguing properties of neural networks. In *ICLR*.

A. Templeton, T. Conerly, et al. 2024. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*.

A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid. 2023. Steering language models with activation engineering. arXiv:2308.10248.

J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. 2020. Investigating gender bias in language models using causal mediation analysis. In *NeurIPS*, vol. 33.

P. Wang, N. Zhang, B. Tian, et al. 2024. EasyEdit: An easy-to-use knowledge editing framework for large language models. In *Proc. ACL (System Demonstrations)*. arXiv:2308.07269.

F. Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts. 2025. AxBench: Steering LLMs? Even simple baselines outperform sparse autoencoders. In *ICML*. arXiv:2501.17148.

C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. 2017. Understanding deep learning requires rethinking generalization. In *ICLR*.

Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, X. He, K.-F. Wong, and P. Minervini. 2025. Steering knowledge selection behaviours in LLMs via SAE-based representation engineering. In *Proc. NAACL*. arXiv:2410.15999.

C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang. 2023. Can we edit factual knowledge by in-context learning? In *Proc. EMNLP*. arXiv:2305.12740.

A. Zou, L. Phan, S. Chen, et al. 2023. Representation engineering: A top-down approach to AI transparency. arXiv:2310.01405.

A Implementation Details

Hyper-parameters. Gradient-descent editor: $K = 200$ candidates, multiplier range $(0, 3)$, margin $\gamma = 0.3$, $\lambda_{\mathrm{KL}} = 3$, $\lambda_{\mathrm{size}} = 0.005$, Adam learning rate 0.1, 100 steps, about 6 to 8 s per prompt; the best iterate is re-verified through the sweep's hook. Sweep: mute strength 0.6 and boost 0.5 (original), mute 1.0 and boost 2.0 (wide). Safety tolerance 10^{-6} in probability. Translator: ridge regression on 500,126 Wikitext-2 token pairs per layer pair, 25 layer pairs in both directions; neural map: one hidden layer. Map trained on edits: 289 successful training edits, dev and test target words held out. Doses are chosen on development records, by the best mean log-rank gain from step D10 on.

Benchmark construction. 21,919 CounterFact records, 16,360 with a single-token true answer ranked between 2 and 1000, subject-disjoint split with five held-out relations (1,000 / 150 / 300), 1,450 cached prompts. The transfer study uses 200 records (50 dev, 150 test, the 100 ROME development records excluded); the edit reaches rank 1 on 116 of them and the 88 primary test records are those among the test records where it does. Import uses 21,789 single-token facts after removing development and pilot records, split into two halves with a fixed seed (the mix weight is chosen on half A, reported on half B).

Compute. All experiments ran on one laptop (RTX 4050, 6.1 GB, 16 CPU cores; torch 2.6.0+cu124, transformer-lens 3.5.1, sae-lens 6.45.3). Medium-sized steps took 20 minutes to about 2 hours; the biggest training runs of step D13 took about 6 hours because a padded training set filled the GPU and slowed the run 5 to 10 times without error (a lesson recorded in the project's rules).

B Pass Bars of the Transfer Study and Their Verdicts

Every bar was written into the project plan, and committed, before the run that judged it; none was moved. "Edit's own" means the effect the source edit has inside Small itself. Verdicts are as printed by the tools.

Step	Bar fixed before the run	Result	Verdict
Gate 1 (A2)	Translator: difference cosine at the last position ≥ 0.3	Import pairs 0.31; Export pair 0.28	Import only
Gate 2 (B1)	Country swap beats the strongest control by a margin, paired test	55% and 33% against $\leq 0.4\%$	pass
Gate 3 (D)	Export top-1 $\geq 15\%$ and 10 points above the strongest control	8%; margin 3 points	fail
Gate 4 (D5)	Best layer set ≥ 10 points above best single layer, $p < 0.01$, top-1 $\geq 15\%$	7% against 8%; $p = 0.016$	fail
Gate 5 (D6)	Top-1 $\geq 50\%$; rewordings lift $\geq 80\%$ of edit's own; neighbours fall ≤ 5 points	95%; 0.91; 8 points	fail (neighbours)
Gate 6 (D7)	Top-1 $\geq 50\%$; rewordings lift ≥ 0.8; neighbours fall no more than edit's own	18%; 0.74; 7 points	fail
Gate 7 (D10)	As Gate 6, plus real target 10 points above random word	20%; 0.74; met; met	fail
Gate 8 (Import)	Rescue $\geq 50\%$ of the 780; accuracy halfway to Medium; 10 points above controls ($p < 0.01$); 90% of known facts kept	33.6%; 7.2% against 9.1%; met; 78.3%	fail
Gate 9 (D13)	Gate 7 unchanged, on the full varied training set	14%; 0.62; met; met	fail
Rule E5	Soft Import: gain ≥ 0.5 points on held-out facts, 90% kept, $p < 0.01$	+1.0 point; 98.3%; $p = 2.5 \times 10^{-14}$	met
Rule S1	SAE basis "matters" if top-1 higher by 10 points or rewordings lift by 5	+15 points; +2 points	matters
Rules S2, S3	Real target's rewordings lift larger than a random word's, $p < 0.01$	$p = 0.031$ (D6); $p = 0.36$ (D7)	not shown

Two notes written after the fact change no verdict: the 5-point neighbour line of Gate 5 was stricter than the source edit's own loss of 9 points inside Small, so Gates 6, 7 and 9 use the edit's own loss; and the dose rule (dev top-1 on 28 records) was replaced by the dev log-rank gain from step D10 on, after step D9b showed it to be fragile.

C Additional Figures

D Reproduction and Where Each Figure Comes From

From the repository root, with `HF_HUB_OFFLINE=1` once the models are cached. The main line is on the branches `layer-intervention-benchmark`, `pool-refill-implementation` and `gradient-descent-editing`; the transfer study is on `cross-model-transfer`.

```
streamlit run experiment_app.py
python run_batch.py data/candidate_source_batch_spec.json
python tools/build_counterfact_set.py
python tools/split_counterfact_set.py
python tools/build_strength_cache.py
python tools/measure_headroom.py
python tools/train_strength_models.py
python tools/compare_sweep_vs_gradient.py --n-per-band 0
python tools/control_random_targets.py
python tools/transfer/00_setup_check.py    (then 01 to 27)
python scripts/rome/06_sweep_layers.py     (ROME environment)
python tools/make_report_figures.py
python tools/make_report_v2_figures.py
python tools/make_report_v3_history_figures.py
python tools/make_report_v4_figures.py
```

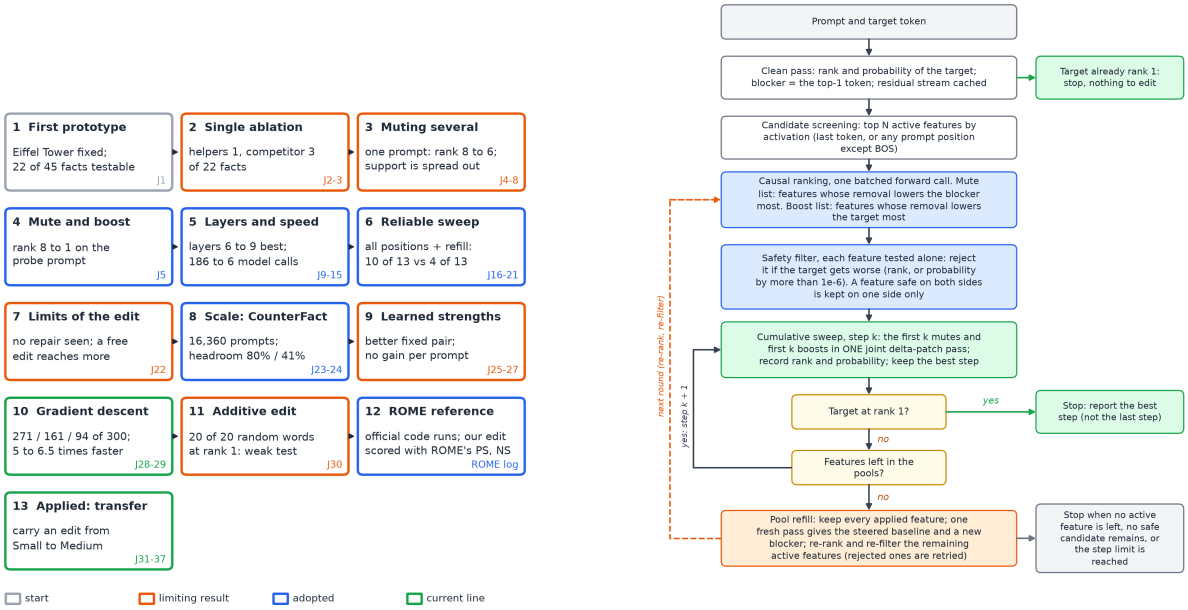


Figure 16: Left: the path of the project in thirteen stages (grey, start; orange, a result that limited the approach; blue, adopted; green, the line current at the time). Right: the hybrid sweep as a flow chart; blue boxes are the ranking and filtering steps of Eqs. 3 and 4, green is the sweep, orange is pool refill.

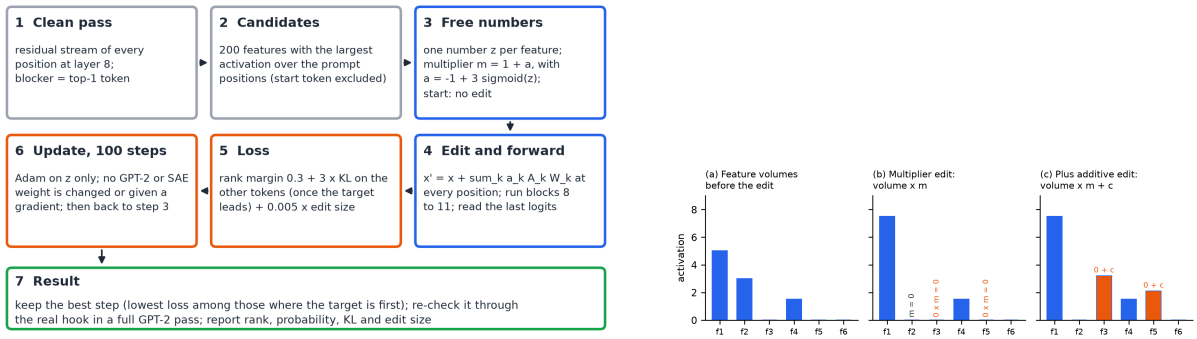


Figure 17: Left: the gradient-descent editor; only the free numbers z_k are updated. Right: why an additive edit was tried; a multiplier leaves a silent feature (f3, f5) at zero, while an additive amount c switches it on.

Every figure is drawn from stored data and no figure script runs a model; diagrams contain no data. The data behind each figure (paths relative to `docs/Research_Journal/` unless stated):

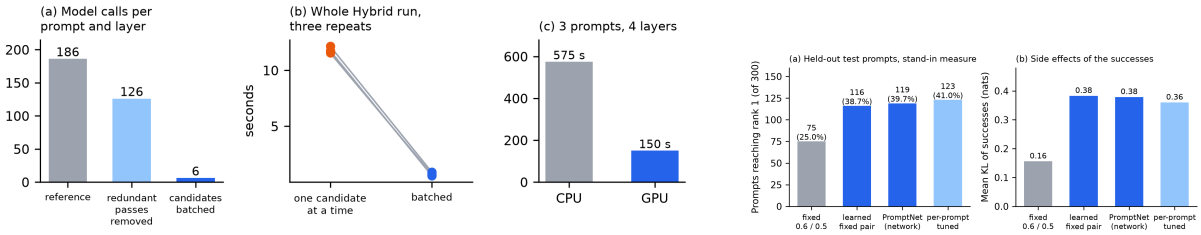


Figure 18: Left: making the computation fast: (a) model calls per prompt and layer, (b) whole hybrid run with the same result, (c) CPU against GPU. Right: learned strengths on the 300 test prompts (stand-in measure): (a) prompts reaching rank 1, (b) mean KL of the successes.

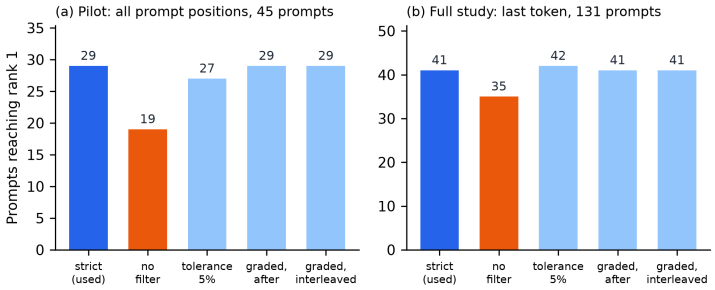


Figure 19: Prompts brought to rank 1 under five versions of the safety filter. (a) 45-prompt pilot, candidates from all prompt positions. (b) 131 prompts, last token only. Switching the filter off loses prompts (29 to 19 in (a)); relaxed versions change the count by at most one.

Figure file	Data	Script
h_stages, h_early, h_speed	Numbers printed in journal entries 1 to 15, typed into the script header	make_report_v3_history_figures
h_gd_flow, h_additive, r_overview	Diagrams, no data	same; make_report_v2_figures
fig_candsrc	benchmark_results/candidate_source_study/	make_report_figures
v4_layer12	benchmark_results/layer_benchmark_20260909_summary.csv	make_report_figures
v4_filter	packs/entry21_*/tables/arms_*.csv	make_report_figures
fig_headroom	packs/headroom/headroom_20261002_144917.json	make_report_figures
fig_learned	packs/strength_models/results.json	make_report_figures
h_gd_bars	images/e29_figure_data.json	make_report_v3_history_figures
v4_gd_side	packs/sweep_vs_gradient/runs.jsonl	make_report_v4_figures
fig_additive, v4_reworded	packs/additive_control/, packs/generalisation/	make_report_figures
v4_rome_trace	data/comparison/trace_summary.json	make_report_v4_figures
v4_rome_sweep	benchmark_results/rome_vs_scalpel/layer_sweep_dev.csv	make_report_v4_figures
v4_rome_vs_ours	that CSV and packs/cross_model_transfer/d_analysis_linear.json	make_report_v4_figures
r_ladder	d5, d6, d7, d10, d13 result files in packs/cross_model_transfer/	make_report_v2_figures
r_push, r_import	s2, s3, e5 result files in the same pack	make_report_v2_figures